Професионална гимназия по високи технологии
“Александър Степанович Попов”

гр. София п.к.1309, бул. Сливница № 10, тел. 02 / 822 11 86, 828 50 10
https://aspopov.bg/, e-mail: info-2206410@edu.mon.bg

ТЕХНИЧЕСКИ ПАСПОРТ

Тема: „Система за дистанционен мониторинг на индустриално оборудване“

Професия код 523030 :Техник на електронна техника
Специалност код 5230302 : „Микропроцесорна техника“ - 12Д Клас

Дипломант:
/Симеон Стоянов Богоев/

Ръководител-Консултант:
/ Евгени Антонов /

София
2025г

Описание на проекта

Това е прототипна IoT система предназначена за дистанционен мониторинг на параметри в индустриална среда , като температура , влага, вибрации, ъгъл на наклон и качество на въздуха. Данните се събират от сензори, свързани към микроконтролер Arduino Uno R4 WiFi, и се предават чрез Wi-Fi към облачна платформа (Arduino IoT Cloud). Системата позволява визуализация на данни в реално време и исторически план, което подпомага ранно откриване на аномалии и оптимизиране на поддръжки.



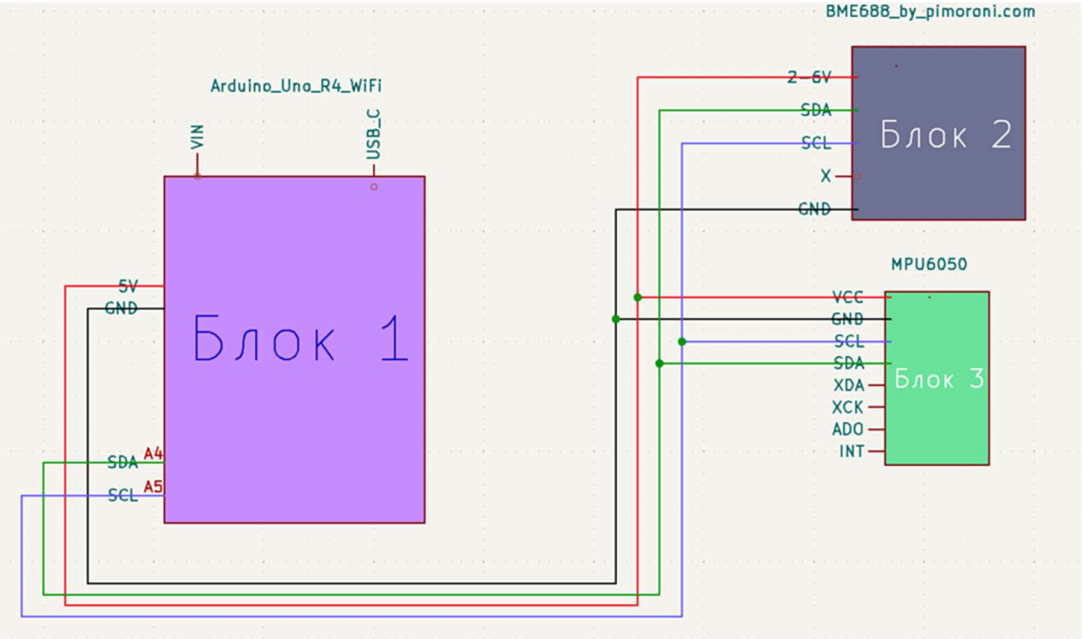
Фиг. 1 Снимка на проекта

Списък на компоненти и технологии

Таблица 1

Компоненти	№	Характеристики	Приложение
Arduino Uno R4 WiFi	1	Микроконтролер: Renesas RA4M1 (48 MHz, ARM Cortex-M4F) Wi-Fi модул: ESP32-S3-MINI-1-N8	Управление на сензори, обработка на данни и комуникация с облака.
Сензор BME688	2	Измерва температура, влажност, качество на въздуха	Мониторинг на въздуха.
Сензор MPU-6050	3	Акселерометър (±16 g), жироскоп (±2000°/s)	Засичане на вибрации и ъгли на наклон.
I2C шина	4	Протокол за комуникация между сензори и микроконтролер	Свързване на BME688 и MPU-6050 към Arduino.
Arduino IoT Cloud	5	Облачна платформа	Визуализация на данни, настройка на прагове и управление на предупреждения.
Захранващ модул	6	12V	Захранва проекта

Блок схема



Фиг. 2 Блок схема на системата

В Фиг. 2 е представено връзките на компонентите, ориентирани по същия начин като в Фиг 1.. Блок 1, Блок 2 и Блок 3 отговарят на компонентите, и техните описания, №1 , № 2 и № 3 в Таблица 1.

Връзките в системата

Таблица 2

Компонент	Пин на компонента	Пин на Arduino	Функция
BME688	SDA	A4	I2C комуникация
	SCL	A5	
	VCC	5V	Захранване
	GND	GND	
MPU-6050	SDA	A4	I2C комуникация
	SCL	A5	
	VCC	5V	Захранване
	GND	GND	

Програмен код

Главния код в проекта в файл с „.ino“ разширение.

```
// 1
#include <Adafruit_MPU6050.h>
#include <Adafruit_BME680.h>
#include "thingProperties.h"
#include <Wire.h>

// 2
Adafruit_MPU6050 mpu;
Adafruit_BME680 bme;

// 3
const float GAS_RESISTANCE_MIN = 1000.0;
const float GAS_RESISTANCE_MAX = 200000.0;
const float AQI_SCALE_MAX = 500.0;

// 4
float ProgramVibrationThreshold = 10.0;
float ProgramTiltThreshold = 10.0;
int ProgramMaxTemp = 100;
int ProgramMinTemp = 0;
int ProgramMaxHumidity = 70;
bool ProgramAQVeryGood = true;
bool ProgramAQNormal = false;
bool ProgramAQVeryBad = false;

// 5
void setup() {
  Serial.begin(9600);
  delay(1500);

  if (!bme.begin(0x76)) {
    Serial.println("Sensor not found at 0x76");
    while (1);
  }
  Serial.println("BME688 Started");

  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1);
  }
```

```
  Serial.println("MPU6050 Started");

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_5_HZ);
  Serial.println("MPU configured");
  delay(100);

  bme.setTemperatureOversampling(BME680_OS_8X);
  bme.setHumidityOversampling(BME680_OS_2X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
  bme.setGasHeater(320, 150);
  Serial.println("BME configured");

  initProperties();
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
  delay(3000);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}

// 6
void loop() {
  ArduinoCloud.update();
  detectVibrationAndTilt();
  delay(500);
  readBmeSensor();
  delay(500);
  invertCloudWarnings();
}

// 7
void detectVibrationAndTilt() {
  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);

  float totalAcceleration = sqrt(accel.acceleration.x * accel.acceleration.x +
                                 accel.acceleration.y * accel.acceleration.y +
                                 accel.acceleration.z * accel.acceleration.z);

  if (totalAcceleration > ProgramVibrationThreshold) {
    cloudVibrationWarning = true;
```

```
  Serial.println("Vibration Detected!");
  Serial.print("Total Acceleration: ");
  Serial.println(totalAcceleration);
 } else {
  cloudVibrationWarning = false;
 }

 float tiltX = atan2(accel.acceleration.y,
                sqrt(accel.acceleration.x * accel.acceleration.x +
                    accel.acceleration.z * accel.acceleration.z)) *
            180 / PI;
 float tiltY = atan2(-accel.acceleration.x,
                sqrt(accel.acceleration.y * accel.acceleration.y +
                    accel.acceleration.z * accel.acceleration.z)) *
            180 / PI;

 cloudTiltX = tiltX;
 cloudTiltY = tiltY;
 cloudLiveVibrations = totalAcceleration;

 static unsigned long tiltStartTime = 0;

 if (abs(tiltX) > ProgramTiltThreshold || abs(tiltY) > ProgramTiltThreshold) {
  cloudTiltWarning = true;
  if (tiltStartTime == 0) {
   tiltStartTime = millis();
  } else if (millis() - tiltStartTime > 500) {
   Serial.println("----------MPU6050----------");
   Serial.println("Tilt Detected!");
   Serial.print("Tilt X: ");
   Serial.println(tiltX);
   Serial.print("Tilt Y: ");
   Serial.println(tiltY);
   Serial.println("----------");

   tiltStartTime = 0;
  }
 } else {
  cloudTiltWarning = false;
  tiltStartTime = 0;
 }
}

// 8
```

```
void readBmeSensor() {
  if (!bme.performReading()) {
    Serial.println("Failed to perform reading");
    return;
  }

  bME_Temp = bme.temperature;
  bME_Index = calculateAQI(bme.gas_resistance, bme.humidity);
  bME_Humidity = bme.humidity;

  raiseAQIndicators();
  raiseTemperatureIndicators();
  raiseHumidityIndicators();
 SerialMonitorDebuggingBME();
}

// 9
void raiseAQIndicators() {
  cloudAQVeryGood = false;
  cloudAQNormal = false;
  cloudAQVeryBad = false;

  if (bME_Index >= 0 && bME_Index <= 100) {
    cloudAQVeryGood = true;
    Serial.println("Air Quality: Very Good");
  } else if (bME_Index > 100 && bME_Index <= 220) {
    cloudAQNormal = true;
    Serial.println("Air Quality: Normal");
  } else if (bME_Index > 220 && bME_Index <= 500) {
    cloudAQVeryBad = true;
    Serial.println("Air Quality: Very Bad");
  } else {
    Serial.println("Air Quality Index out of range");
  }
}

// 10
void raiseTemperatureIndicators() {
  cloudTempWarning = false;
  cloudTooLowTempWarning = false;

  if (bME_Temp > ProgramMaxTemp) {
    cloudTempWarning = true;
    Serial.println("WARNING: Temperature exceeded the upper limit!");
```

```
  } else if (bME_Temp < ProgramMinTemp) {
    cloudTooLowTempWarning = true;
    Serial.println("WARNING: Temperature fell below the lower limit!");
  } else {
    Serial.println("Temperature is within normal limits.");
  }
}

// 11
void raiseHumidityIndicators() {
  cloudHumidityWarning = false;

  if (bME_Humidity > ProgramMaxHumidity) {
    cloudHumidityWarning = true;
    Serial.println("WARNING: Humidity is above the configured upper limit!");
  } else {
    Serial.println("Humidity is within the configured range.");
  }
}

// 12
void SerialMonitorDebuggingBME() {
  Serial.println("--------------BME688--------------------");
  Serial.print("Temperature: ");
  Serial.print(bme.temperature);
  Serial.println(" °C");
  Serial.print("Humidity: ");
  Serial.print(bme.humidity);
  Serial.println(" %");
  Serial.print("Pressure: ");
  Serial.print(bme.pressure / 100.0);
  Serial.println(" hPa");
  Serial.print("Gas Resistance: ");
  Serial.print(bme.gas_resistance / 1000.0);
  Serial.println(" kOhm");
  Serial.println("---------------------------------");
}

// 13
void invertCloudWarnings() {
  cloudTempWarning = !cloudTempWarning;
  cloudTiltWarning = !cloudTiltWarning;
  cloudTooLowTempWarning = !cloudTooLowTempWarning;
  cloudVibrationWarning = !cloudVibrationWarning;
```

```
  cloudHumidityWarning = !cloudHumidityWarning;
}

// 14
float calculateAQI(float gasResistance, float humidity) {
  float aqi = 0.0;

  float gasScore = (log(gasResistance) - log(GAS_RESISTANCE_MIN)) /
                   (log(GAS_RESISTANCE_MAX) - log(GAS_RESISTANCE_MIN));
  gasScore = constrain(gasScore, 0, 1);
  gasScore = gasScore * AQI_SCALE_MAX;

  float humidityScore = 0;
  if (humidity > ProgramMaxHumidity) {
    humidityScore = (humidity - ProgramMaxHumidity) / ProgramMaxHumidity;
  }
  humidityScore = humidityScore * AQI_SCALE_MAX;

  aqi = (gasScore + humidityScore) / 2;

  return aqi;
}

// 15
void onCloudTempUpperLimitInputChange() {
  if (cloudTemp_Upper_Limit_input != ProgramMaxTemp) {
    ProgramMaxTemp = cloudTemp_Upper_Limit_input;
    Serial.println("TempUpperLimitChanged");
  }
}

// 16
void onCloudTempLowerLimitInputChange() {
  if (cloudTemp_Lower_Limit_input != ProgramMinTemp) {
    ProgramMinTemp = cloudTemp_Lower_Limit_input;
    Serial.println("TempLowerLimitChanged");
  }
}

// 17
void onCloudVibrationUpperLimitInputChange() {
  if (cloud_Vibration_Upper_Limit_input != ProgramVibrationThreshold) {
    ProgramVibrationThreshold = cloud_Vibration_Upper_Limit_input;
    Serial.println("VibrationUpperLimitChanged");
```

```
  }
}

// 18
void onCloudHumidityUpperLimitInputChange() {
  if (cloudHumidity_Upper_Limit_input != ProgramMaxHumidity) {
    ProgramMaxHumidity = cloudHumidity_Upper_Limit_input;
    Serial.println("HumidityUpperLimitChanged");
  }
}

// 19
void onCloudTiltLimitInputChange() {
  if (cloudTilt_Limit_input != ProgramTiltThreshold) {
    ProgramTiltThreshold = cloudTilt_Limit_input;
    Serial.println("TiltLimitChanged");
  }
}
```

Код, генериран от средата за разработване на облачната платформа, в файл “thingProperties.h“. Там също се записва име и парола за локална мрежа.

```
// Code generated by Arduino IoT Cloud, DO NOT EDIT.

#include <ArduinoIoTCloud.h>
#include <Arduino_ConnectionHandler.h>

const char SSID[]     = SECRET_SSID;    // Network SSID (name)
const char PASS[]     = SECRET_OPTIONAL_PASS;    // Network password (use for
WPA, or use as key for WEP)

void onCloudTiltLimitInputChange();
void onCloudVibrationUpperLimitInputChange();
void onCloudHumidityUpperLimitInputChange();
void onCloudTempLowerLimitInputChange();
void onCloudTempUpperLimitInputChange();

float bME_Index;
float cloudLiveVibrations;
float cloudTilt_Limit_input;
float cloudTiltX;
float cloudTiltY;
```

```
float cloud_Vibration_Upper_Limit_input;

CloudTemperatureSensor bME_Temp;
int bME_Humidity;
int cloudHumidity_Upper_Limit_input;
int cloudTemp_Lower_Limit_input;
int cloudTemp_Upper_Limit_input;
bool cloudAQNormal;
bool cloudAQVeryBad;
bool cloudAQVeryGood;
bool cloudHumidityWarning;
bool cloudTempWarning;
bool cloudTiltWarning;
bool cloudTooLowTempWarning;
bool cloudVibrationWarning;

void initProperties(){

  ArduinoCloud.addProperty(bME_Index, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(cloudLiveVibrations, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(cloudTilt_Limit_input, READWRITE, ON_CHANGE,
onCloudTiltLimitInputChange);
  ArduinoCloud.addProperty(cloudTiltX, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(cloudTiltY, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(cloud_Vibration_Upper_Limit_input, READWRITE,
ON_CHANGE, onCloudVibrationUpperLimitInputChange);
  ArduinoCloud.addProperty(bME_Temp, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(bME_Humidity, READ, 1 * SECONDS, NULL);
  ArduinoCloud.addProperty(cloudHumidity_Upper_Limit_input, READWRITE,
ON_CHANGE, onCloudHumidityUpperLimitInputChange);
  ArduinoCloud.addProperty(cloudTemp_Lower_Limit_input, READWRITE,
ON_CHANGE, onCloudTempLowerLimitInputChange);

  ArduinoCloud.addProperty(cloudTemp_Upper_Limit_input, READWRITE,
ON_CHANGE, onCloudTempUpperLimitInputChange);

  ArduinoCloud.addProperty(cloudAQNormal, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudAQVeryBad, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudAQVeryGood, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudHumidityWarning, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudTempWarning, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudTiltWarning, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(cloudTooLowTempWarning, READ, ON_CHANGE,
NULL);
```

```
  ArduinoCloud.addProperty(cloudVibrationWarning, READ, ON_CHANGE, NULL);



}

WiFiConnectionHandler ArduinoIoTPreferredConnection(SSID, PASS);
```

“arduino_secrets.h” файл създаден от платформата.

```
#define SECRET_OPTIONAL_PASS ""
#define SECRET_SSID ""
```

„sketch.json” файл създаден от платформата.

```
{
  "cpu": {
    "fqbn": "arduino:renesas_uno:unor4wifi",
    "name": "",
    "type": "serial"
  },
  "secrets": [
    {
      "name": "SECRET_OPTIONAL_PASS",
      "value": ""
    },
    {
      "name": "SECRET_SSID",
      "value": ""
    }
  ],
  "included_libs": []
}
```

Инструкции за работа

1. Изтеглете от Google Play Store апликацията Arduino IoT Remote

2. Инсталирайте я.

3. Влезте в апликацията с Login Name: DPUser и Password: Service1!

4. Заредете таблото на наблюдавана индустриална среда: “Simeon_Bogoev_12D_22

5. На екрана ще виждате, графично изобразени, наблюдаваните параметри от наблюдаваната индустриална среда. В таблото „Simeon_Bogoev_12D_22“ индикаторите със зелен флаг показват, че параметрите са в нормите зададени, като нормални. Червеният флаг, съответно, показва че, има задействана аларма и че, един или повече параметри са излезли извън от програмираните, като нормални граници.

6. Другото табло „Graphs” има графично изобразяване на параметрите от таблото „Simeon_Bogoev_12D_22“

Нужно е локална мрежа да в обхвата на проекта с име „ДипломенПроект“ и парола „admin123”.